

HDOC DAY - 20

QUESTION:

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then

develop, compile, and execute a menu-driven modular C project that calculates the geometrical measurements

and physical properties of selected solids. Test the program using the prepared test cases.

[Note: Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use

recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, typedef, or functions

from math.h. Assume that all input values and calculated results are within the ranges supported by their respective

data types.]

Operations

- Find the total surface area and volume of a sphere [double sphere_area(double radius), double

sphere_volume(double radius)]

- Find the total surface area and volume of a cylinder [double cylinder_area(double radius, double height),

double cylinder_volume(double radius, double height)]

- Find the total surface area and volume of a cone [double cone_area(double radius, double height), double

cone_volume(double radius, double height)]

- Find the total surface area and volume of a cuboid [double cuboid_area(double length, double breadth,

double height), double cuboid_volume(double length, double breadth, double height)]

- Find the base area of cylinder and cone [double circular_base_area(double radius)]

- Find the base area of cuboid [double cuboid_base_area(double length, double breadth)]
- Calculate the mass of a solid from its density and volume [double calculate_mass(double density)]
- Calculate the weight of a solid [double calculate_weight()]
- Calculate the gravitational potential energy of a solid at a given elevation [double potential_energy(double elevation)]
- Calculate the pressure exerted by a cylinder, cone, or cuboid resting on its base [double calculate_pressure(double density)]
- Calculate the buoyant force on a completely submerged solid [double buoyant_force(double fluid_density)]
- Determine whether a completely immersed solid sinks, floats, or remains suspended [int determine_buoyancy(double solid_density, double fluid_density)]

Aim:

To develop a menu-driven modular C program to calculate geometrical measurements and physical properties of different solids such as sphere, cylinder, cone, and cuboid.

The program calculates:

1. Surface Area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational Potential Energy
7. Pressure on Base
8. Buoyant Force

9. Buoyancy Behaviour

10. Exit

Algorithms:

Algorithm 1: Main Program

1. Start.
2. Display the main menu.
3. Enter the user's choice.
4. Check whether the choice is between 1 and 10.
5. If invalid, display an error and ask again.
6. Use switch-case according to the selected operation.
7. For geometrical operations, ask the user to select the solid.
8. Accept the required dimensions.
9. Validate that dimensions are greater than zero.
10. Call the appropriate function.
11. Display the result with the correct SI unit.
12. Repeat the menu using a do-while loop.
13. If choice is 10, terminate the program.
14. Stop.

Algorithm 2: Surface Area

1. Start.
2. Select the solid.
3. Enter required dimensions.
4. Validate the dimensions.
5. For sphere:

$\text{Area} = 4 \times \text{PI} \times \text{radius} \times \text{radius}$

6. For cylinder:

$\text{Area} = 2 \times \text{PI} \times \text{radius} \times (\text{radius} + \text{height})$

7. For cone, calculate slant height using the square-root function and then:

Area = $\text{PI} \times \text{radius} \times (\text{radius} + \text{slant height})$

8. For cuboid:

Area = $2 \times (\text{length} \times \text{breadth} + \text{breadth} \times \text{height} + \text{length} \times \text{height})$

9. Return the calculated area.

10. Stop.

Algorithm 3: Volume

1. Start.

2. Select the solid.

3. Enter and validate dimensions.

4. Calculate volume according to the solid.

5. Sphere:

Volume = $(4/3) \times \text{PI} \times r^3$

6. Cylinder:

Volume = $\text{PI} \times r^2 \times h$

7. Cone:

Volume = $(1/3) \times \text{PI} \times r^2 \times h$

8. Cuboid:

Volume = $\text{length} \times \text{breadth} \times \text{height}$

9. Return volume.

10. Stop.

Algorithm 4: Base Area

1. Start.

2. Select cylinder, cone, or cuboid.

3. If sphere is selected, display that base area cannot be calculated.

4. For cylinder/cone:

Base Area = $\text{PI} \times \text{radius} \times \text{radius}$

5. For cuboid:

Base Area = $\text{length} \times \text{breadth}$

6. Return base area.
7. Stop.

Algorithm 5: Mass

1. Start.
2. Enter density.
3. Validate density so that it is greater than zero.
4. Select the solid.
5. Enter its dimensions.
6. Calculate its volume using the appropriate volume function.
7. Calculate:

$$\text{Mass} = \text{Density} \times \text{Volume}$$

8. Return mass.
9. Stop.

Algorithm 6: Weight

1. Start.
2. Obtain mass using `calculate_mass()`.
3. Calculate:

$$\text{Weight} = \text{Mass} \times \text{GRAVITY}$$

4. Return weight.
5. Stop.

Algorithm 7: Gravitational Potential Energy

1. Start.
2. Enter elevation.
3. Elevation must be greater than or equal to zero.
4. Obtain mass using `calculate_mass()`.
5. Calculate:

$$\text{Potential Energy} = \text{Mass} \times \text{GRAVITY} \times \text{Elevation}$$

6. Return potential energy.

7. Stop.

Algorithm 8: Pressure

1. Start.
2. Enter density.
3. Select cylinder, cone, or cuboid.
4. Enter required dimensions.
5. Calculate volume.
6. Calculate base/contact area.
7. Calculate:

$$\text{Pressure} = (\text{Density} \times \text{Volume} \times \text{GRAVITY}) / \text{Contact Area}$$

8. Return pressure.
9. Stop.

Algorithm 9: Buoyant Force

1. Start.
2. Enter fluid density.
3. Select the solid.
4. Enter required dimensions.
5. Calculate volume.
6. Calculate:

$$\text{Buoyant Force} = \text{Fluid Density} \times \text{Volume} \times \text{GRAVITY}$$

7. Return buoyant force.
8. Stop.

Algorithm 10: Buoyancy Behaviour

1. Start.
2. Enter solid density.
3. Enter fluid density.
4. Compare both densities.
5. If solid density > fluid density, return SINKS.

6. If solid density < fluid density, return FLOATS.
7. Otherwise, return SUSPENDED.
8. Stop.

Algorithm 11: Square Root Using Newton-Raphson

1. Start.
2. Receive number.
3. If number is negative, return -1.
4. If number is zero, return 0.
5. Set old_value = number.
6. Calculate:

$\text{new_value} = 0.5 \times (\text{old_value} + \text{number} / \text{old_value})$

7. Find the difference between new and old values.
8. If difference is negative, make it positive.
9. Assign new_value to old_value.
10. Repeat until difference becomes less than TOLERANCE.
11. Return new_value.
12. Stop.

HEADER FILE:

C PROGRAM:

```
#ifndef SOLID_H
```

```
#define SOLID_H
```

```
#define PI 3.1416
```

```
#define GRAVITY 9.8
```

```
#define TOLERANCE 0.000001
```

```
#define SPHERE 1
```

```
#define CYLINDER 2
```

```
#define CONE 3
```

```
#define CUBOID 4
```

```
#define FLOATS -1
```

```
#define SUSPENDED 0
```

```
#define SINKS 1
```

```
/* Geometry functions */
```

```
double sphere_area(double radius);
```

```
double sphere_volume(double radius);
```

```
double cylinder_area(double radius, double height);
```

```
double cylinder_volume(double radius, double height);
```

```
double cone_area(double radius, double height);
```

```
double cone_volume(double radius, double height);
```

```
double cuboid_area(double length, double breadth, double height);
```

```
double cuboid_volume(double length, double breadth, double height);
```

```
double circular_base_area(double radius);
```

```
double cuboid_base_area(double length, double breadth);
```

```
/* Physics functions */
```

```
double calculate_mass(double density);
```

```
double calculate_weight();
```

```
double potential_energy(double elevation);
```

```
double calculate_pressure(double density);  
double buoyant_force(double fluid_density);  
int determine_buoyancy(double solid_density, double fluid_density);
```

```
/* Mathematical function */
```

```
double calculate_square_root(double number);
```

```
#endif
```

GEOMETRY MODULE:

C PROGRAM:

```
#include "solid.h"
```

```
/* Calculate total surface area of sphere */
```

```
double sphere_area(double radius)
```

```
{  
    return 4 * PI * radius * radius;  
}
```

```
/* Calculate volume of sphere */
```

```
double sphere_volume(double radius)
```

```
{  
    return (4.0 / 3.0) * PI * radius * radius * radius;  
}
```

```
/* Calculate total surface area of cylinder */
```

```
double cylinder_area(double radius, double height)
```

```
{
```

```
    return 2 * PI * radius * (radius + height);  
}
```

```
/* Calculate volume of cylinder */
```

```
double cylinder_volume(double radius, double height)  
{  
    return PI * radius * radius * height;  
}
```

```
/* Calculate total surface area of cone */
```

```
double cone_area(double radius, double height)  
{  
    double slant_height;  
  
    slant_height =  
        calculate_square_root(radius * radius + height * height);  
  
    return PI * radius * (radius + slant_height);  
}
```

```
/* Calculate volume of cone */
```

```
double cone_volume(double radius, double height)  
{  
    return (1.0 / 3.0) * PI * radius * radius * height;  
}
```

```
/* Calculate total surface area of cuboid */
```

```
double cuboid_area(double length, double breadth, double height)
```

```
{  
    return 2 * (length * breadth +  
                breadth * height +  
                length * height);  
}
```

/* Calculate volume of cuboid */

```
double cuboid_volume(double length, double breadth, double height)  
{  
    return length * breadth * height;  
}
```

/* Calculate circular base area */

```
double circular_base_area(double radius)  
{  
    return PI * radius * radius;  
}
```

/* Calculate cuboid base area */

```
double cuboid_base_area(double length, double breadth)  
{  
    return length * breadth;  
}
```

SQUARE ROOT MODULE:

C PROGRAM:

```
#include "solid.h"
```

```
/* Calculate square root using Newton-Raphson method */  
double calculate_square_root(double number)  
{  
    double old_value, new_value, difference;  
  
    if (number < 0)  
        return -1;  
  
    if (number == 0)  
        return 0;  
  
    old_value = number;  
  
    do  
    {  
        new_value =  
            0.5 * (old_value + number / old_value);  
  
        difference = new_value - old_value;  
  
        if (difference < 0)  
            difference = -difference;  
  
        old_value = new_value;  
  
    } while (difference >= TOLERANCE);  
  
    return new_value;
```

```
}
```

PHYSICS MODULE:

C PROGRAM:

```
#include <stdio.h>
```

```
#include "solid.h"
```

```
/* Calculate mass of a solid */
```

```
double calculate_mass(double density)
```

```
{
```

```
    int solid;
```

```
    double radius, height, length, breadth;
```

```
    double volume = 0;
```

```
    do
```

```
    {
```

```
        printf("\nSelect Solid:\n");
```

```
        printf("1. Sphere\n");
```

```
        printf("2. Cylinder\n");
```

```
        printf("3. Cone\n");
```

```
        printf("4. Cuboid\n");
```

```
        printf("Enter choice: ");
```

```
        scanf("%d", &solid);
```

```
        if (solid < 1 || solid > 4)
```

```
            printf("Invalid solid choice.\n");
```

```
    } while (solid < 1 || solid > 4);
```

```
switch (solid)
{
    case SPHERE:

        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);

            if (radius <= 0)
                printf("Radius must be greater than zero.\n");

        } while (radius <= 0);

        volume = sphere_volume(radius);
        break;

    case CYLINDER:

        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        do
        {
            printf("Enter height (m): ");
```

```
scanf("%lf", &height);  
} while (height <= 0);
```

```
volume = cylinder_volume(radius, height);  
break;
```

case CONE:

```
do  
{  
    printf("Enter radius (m): ");  
    scanf("%lf", &radius);  
} while (radius <= 0);
```

```
do  
{  
    printf("Enter height (m): ");  
    scanf("%lf", &height);  
} while (height <= 0);
```

```
volume = cone_volume(radius, height);  
break;
```

case CUBOID:

```
do  
{  
    printf("Enter length (m): ");
```

```

        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
    break;
}

return density * volume;
}

/* Calculate weight */
double calculate_weight()
{
    double density;
    double mass;

```

```

do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater than zero.\n");

} while (density <= 0);

mass = calculate_mass(density);

return mass * GRAVITY;
}

```

```

/* Calculate gravitational potential energy */
double potential_energy(double elevation)
{
    double density;
    double mass;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);
    }
    while (density <= 0);

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

```

```
        if (density <= 0)
            printf("Density must be greater than zero.\n");

    } while (density <= 0);

    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}
```

```
/* Calculate pressure */
double calculate_pressure(double density)
{
    int solid;

    double radius, height, length, breadth;

    double volume = 0;

    double contact_area = 0;

    do
    {
        printf("\nSelect Solid:\n");

        printf("2. Cylinder\n");

        printf("3. Cone\n");

        printf("4. Cuboid\n");

        printf("Enter choice: ");

        scanf("%d", &solid);
```

```

if (solid < 2 || solid > 4)

    printf("Pressure cannot be calculated for this choice.\n");

} while (solid < 2 || solid > 4);

switch (solid)
{
    case CYLINDER:

        do

        {
            printf("Enter radius (m): ");

            scanf("%lf", &radius);

        } while (radius <= 0);

        do

        {
            printf("Enter height (m): ");

            scanf("%lf", &height);

        } while (height <= 0);

        volume = cylinder_volume(radius, height);

        contact_area = circular_base_area(radius);

        break;

    case CONE:

        do

```

```
{  
    printf("Enter radius (m): ");  
    scanf("%lf", &radius);  
} while (radius <= 0);  
  
do  
{  
    printf("Enter height (m): ");  
    scanf("%lf", &height);  
} while (height <= 0);  
  
volume = cone_volume(radius, height);  
contact_area = circular_base_area(radius);  
break;
```

case CUBOID:

```
do  
{  
    printf("Enter length (m): ");  
    scanf("%lf", &length);  
} while (length <= 0);  
  
do  
{  
    printf("Enter breadth (m): ");  
    scanf("%lf", &breadth);  
} while (breadth <= 0);
```

```

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);

volume = cuboid_volume(length, breadth, height);
contact_area = cuboid_base_area(length, breadth);
break;
}

return (density * volume * GRAVITY) / contact_area;
}

```

```

/* Calculate buoyant force */
double buoyant_force(double fluid_density)
{
    int solid;
    double radius, height, length, breadth;
    double volume = 0;

do
{
    printf("\nSelect Solid:\n");
    printf("1. Sphere\n");
    printf("2. Cylinder\n");

```

```
printf("3. Cone\n");
printf("4. Cuboid\n");
printf("Enter choice: ");
scanf("%d", &solid);

} while (solid < 1 || solid > 4);

switch (solid)
{
    case SPHERE:

        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        volume = sphere_volume(radius);
        break;

    case CYLINDER:

        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);
```

```
do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);

volume = cylinder_volume(radius, height);

break;
```

case CONE:

```
do
{
    printf("Enter radius (m): ");
    scanf("%lf", &radius);
} while (radius <= 0);
```

```
do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);
```

```
volume = cone_volume(radius, height);

break;
```

case CUBOID:

```

do
{
    printf("Enter length (m): ");
    scanf("%lf", &length);
} while (length <= 0);

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
} while (breadth <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);

volume = cuboid_volume(length, breadth, height);
break;
}

return fluid_density * volume * GRAVITY;
}

/* Determine buoyancy behaviour */
int determine_buoyancy(double solid_density,

```

```

        double fluid_density)
{
    if (solid_density > fluid_density)

        return SINKS;

    else if (solid_density < fluid_density)

        return FLOATS;

    else

        return SUSPENDED;
}

```

MAIN PROGRAM:

C PROGRAM:

```
#include <stdio.h>
```

```
#include "solid.h"
```

```
int main()
```

```

{
    int choice, solid, result;

    double radius, height, length, breadth;

    double density, fluid_density, solid_density;

    double elevation, answer;

    do
    {
        printf("\n===== MAIN MENU =====\n");

        printf("1. Surface Area\n");

        printf("2. Volume\n");

        printf("3. Base Area\n");

        printf("4. Mass\n");
    }
}

```

```
printf("5. Weight\n");  
printf("6. Gravitational Potential Energy\n");  
printf("7. Pressure on Base\n");  
printf("8. Buoyant Force\n");  
printf("9. Buoyancy Behaviour\n");  
printf("10. Exit\n");  
printf("=====\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
if (choice < 1 || choice > 10)  
{  
    printf("Invalid menu choice.\n");  
    continue;  
}
```

```
switch (choice)  
{  
    case 1:  
    case 2:  
    case 3:  
  
        do  
        {  
            printf("\nSelect Solid:\n");  
            printf("1. Sphere\n");  
            printf("2. Cylinder\n");
```

```

printf("3. Cone\n");
printf("4. Cuboid\n");
printf("Enter choice: ");
scanf("%d", &solid);

if (solid < 1 || solid > 4)
    printf("Invalid solid choice.\n");

} while (solid < 1 || solid > 4);

if (choice == 3 && solid == SPHERE)
{
    printf("Base area of sphere is not defined.\n");
    break;
}

if (solid == SPHERE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);

        if (radius <= 0)
            printf("Radius must be greater than zero.\n");

    } while (radius <= 0);
}

```

```
    if (choice == 1)
        answer = sphere_area(radius);
    else
        answer = sphere_volume(radius);
}

else if (solid == CYLINDER)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    if (choice != 3)
    {
        do
        {
            printf("Enter height (m): ");
            scanf("%lf", &height);
        } while (height <= 0);
    }

    if (choice == 1)
        answer = cylinder_area(radius, height);
    else if (choice == 2)
        answer = cylinder_volume(radius, height);
    else
```

```
        answer = circular_base_area(radius);
    }

    else if (solid == CONE)
    {
        do
        {
            printf("Enter radius (m): ");
            scanf("%lf", &radius);
        } while (radius <= 0);

        if (choice != 3)
        {
            do
            {
                printf("Enter height (m): ");
                scanf("%lf", &height);
            } while (height <= 0);
        }

        if (choice == 1)
            answer = cone_area(radius, height);
        else if (choice == 2)
            answer = cone_volume(radius, height);
        else
            answer = circular_base_area(radius);
    }
```

```
else  
  
{  
    do  
    {  
        printf("Enter length (m): ");  
        scanf("%lf", &length);  
    } while (length <= 0);  
  
    do  
    {  
        printf("Enter breadth (m): ");  
        scanf("%lf", &breadth);  
    } while (breadth <= 0);  
  
    if (choice != 3)  
    {  
        do  
        {  
            printf("Enter height (m): ");  
            scanf("%lf", &height);  
        } while (height <= 0);  
    }  
  
    if (choice == 1)  
        answer = cuboid_area(length,  
                                breadth,  
                                height);  
    else if (choice == 2)
```

```
        answer = cuboid_volume(length,
                                breadth,
                                height);
    else
        answer = cuboid_base_area(length,
                                    breadth);
}
```

```
if (choice == 1 || choice == 3)
    printf("Area = %.4lf square metres\n",
           answer);
else
    printf("Volume = %.4lf cubic metres\n",
           answer);
```

```
break;
```

case 4:

```
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);

    if (density <= 0)
        printf("Density must be greater than zero.\n");
```

```
} while (density <= 0);
```

```
answer = calculate_mass(density);
```

```
printf("Mass = %.4lf kg\n", answer);
```

```
break;
```

case 5:

```
answer = calculate_weight();
```

```
printf("Weight = %.4lf N\n", answer);
```

```
break;
```

case 6:

```
do
```

```
{
```

```
    printf("Enter elevation (m): ");
```

```
    scanf("%lf", &elevation);
```

```
    if (elevation < 0)
```

```
        printf("Elevation cannot be negative.\n");
```

```
} while (elevation < 0);
```

```
answer = potential_energy(elevation);
```

```
printf("Potential Energy = %.4lf J\n",
```

```
    answer);
```

```
break;
```

```
case 7:
```

```
do
```

```
{
```

```
    printf("Enter density (kg/m^3): ");
```

```
    scanf("%lf", &density);
```

```
    if (density <= 0)
```

```
        printf("Density must be greater than zero.\n");
```

```
    } while (density <= 0);
```

```
answer = calculate_pressure(density);
```

```
printf("Pressure = %.4lf Pa\n", answer);
```

```
break;
```

```
case 8:
```

```
do
```

```
{  
    printf("Enter fluid density (kg/m^3): ");  
    scanf("%lf", &fluid_density);  
  
    if (fluid_density <= 0)  
        printf("Density must be greater than zero.\n");  
  
} while (fluid_density <= 0);  
  
answer = buoyant_force(fluid_density);  
  
printf("Buoyant Force = %.4lf N\n",  
    answer);  
break;
```

case 9:

```
do  
{  
    printf("Enter solid density (kg/m^3): ");  
    scanf("%lf", &solid_density);  
  
} while (solid_density <= 0);  
  
do  
{  
    printf("Enter fluid density (kg/m^3): ");
```

```
        scanf("%lf", &fluid_density);

    } while (fluid_density <= 0);

    result =
        determine_buoyancy(solid_density,
            fluid_density);

    if (result == SINKS)
        printf("The solid SINKS.\n");

    else if (result == FLOATS)
        printf("The solid FLOATS.\n");

    else
        printf("The solid remains SUSPENDED.\n");

    break;

case 10:
    printf("Program terminated.\n");
    break;
}

} while (choice != 10);

return 0;
```

}

TEST CASE TABLE:

TEST	OPERATIONS	INPUT	Type	Expected result
1	Sphere area	r = 2	Valid	50.2656 m ²
2	Sphere volume	r = 3	Valid	113.0976 m ³
3	Cylinder area	r=2, h=5	Valid	87.9648 m ²
4	Cylinder volume	r=2, h=5	Valid	62.8320 m ³
5	Cone volume	r=3, h=4	Valid	37.6992 m ³
6	Cuboid area	l=2,b=3,h=4	Valid	52 m ²
7	Cuboid volume	l=2,b=3,h=4	Valid	24 m ³
8	Cylinder Base	r=2	Valid	12.5664 m ²
9	Sphere Base	Sphere	Invalid operation	Error/message
10	Radius	0	Boundary/Invalid	Ask again
11	Radius	-2	Invalid	Ask again
12	Height	0	Boundary/Invalid	Ask again
13	Density	-100	Invalid	Ask again
14	Elevation	-5	Invalid	Ask again
15	Elevation	0	Boundary/Valid	Ask again
16	Solid choice	0	Invalid	Ask again
17	Solid choice	5	Invalid	Ask again
18	Menu choice	0	Invalid	Error message
19	Menu choice	11	Invalid	Error message
20	Buoyancy	solid=800, fluid=1000	Valid	FLOATS
21	Buoyancy	solid=1200, fluid=1000	Valid	SINKS
22	Buoyancy	solid=1000, fluid=1000	Boundary	SUSPENDED
23	Exit	Choice = 10	Valid	Program terminates

Result:

Thus, a menu-driven modular C program was developed to calculate the geometrical measurements and physical properties of sphere, cylinder, cone, and cuboid. The program uses separate modules, functions with call by value, switch-case, loops, input validation, and the Newton-Raphson method for square-root calculation as required by the assignment.

